

PROGRAMACIÓN IV — UTN TUP A Distancia

Guía Paso a Paso: Segundo Parcial

Auth · RBAC · Pedidos · Carrito · Administración

Esta guía asume que el Parcial 1 (Producto, Categoría, Ingrediente) ya está funcionando. Cada sección explica el QUÉ, el CÓMO y el POR QUÉ de los nuevos conceptos.

Mapa del Proyecto: Qué hay de nuevo respecto al Parcial 1

El Parcial 2 agrega tres grandes bloques sobre la base ya construida: **autenticación con JWT**, un **sistema de roles (RBAC)**, y la **lógica de pedidos**. Además, el frontend se divide en dos aplicaciones separadas que consumen el mismo backend.

```
BACKEND (FastAPI + PostgreSQL)
├─ [PARCIAL 1] Categoría, Producto, Ingrediente (ya hecho)
├─ [NUEVO] Auth: registro, login, JWT en cookie, /me
├─ [NUEVO] Roles: Usuario, Rol, UsuarioRol → RBAC
├─ [NUEVO] Pedidos: Pedido, DetallePedido, EstadoPedido
├─ [NUEVO] Historial: HistorialEstadoPedido (append-only)
├─ [NUEVO] FormaPago, DireccionEntrega
└─ [NUEVO] Patrones: UnitOfWork, Repository, SoftDelete

FRONTEND A – Store (React + Zustand + TanStack Query)
├─ Home del catálogo
├─ Carrito con persistencia en localStorage (Zustand persist)
└─ Flujo de pedido: confirmar → ver estado

FRONTEND B – Administración (React + TanStack Query)
├─ Login → Cookie JWT → Rutas protegidas por rol
├─ Pantallas del Parcial 1 (Categoría, Producto, Ingrediente)
└─ Pantalla Cajero: avanzar estados de pedidos
```

Principio clave: los dos frontends son proyectos Vite separados, pero comparten el mismo backend. Cada uno tiene su instancia de Axios con su propia configuración de base URL e interceptores.

PARTE 1 — Conceptos Nuevos Explicados

1.1 — JWT (JSON Web Token): qué es y cómo funciona

Un **JWT** es una cadena de texto firmada digitalmente que el servidor emite cuando el usuario se loguea correctamente. Funciona como un **documento de identidad temporal**: el cliente lo guarda y lo presenta en cada request para demostrar quién es, sin que el servidor tenga que consultar la base de datos en cada pedido.

Un JWT tiene tres partes separadas por puntos:

```
eyJhbGciOiJIUzI1NiJ9      <- Header: algoritmo usado
.eyJ1c2VyX2lkIjoxLCJyb2wiOiJBRE1JTj9  <- Payload: datos (claims)
.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV  <- Signature: verificación de integridad
```

El payload puede contener cualquier dato: `user_id`, `rol`, `exp` (fecha de expiración). La firma garantiza que nadie lo modificó. El parcial usa **cookies HttpOnly** para guardarlo, lo cual es más seguro que `localStorage` porque JavaScript del browser no puede leerlas.

 ¿Por qué cookie `HttpOnly` y no `localStorage`? Porque si hay un ataque XSS (inyección de JS malicioso), ese script no puede robar la cookie. Con `localStorage` sí podría. Es la práctica recomendada para tokens de sesión.

1.2 — RBAC: Control de Acceso Basado en Roles

RBAC (Role-Based Access Control) es el patrón donde los permisos no se asignan a usuarios directamente, sino a **roles**, y los usuarios reciben roles. Así un ADMIN puede hacer todo, un empleado PEDIDOS solo puede ver y avanzar pedidos, y un CLIENT solo ve sus propias cosas.

Rol	Qué puede hacer
ADMIN	CRUD completo de todo el sistema
STOCK	Leer productos, actualizar stock y disponibilidad
PEDIDOS	Ver y avanzar estados de pedidos (el 'cajero')
CLIENT	Catálogo público, carrito, sus propios pedidos

En FastAPI implementás RBAC con **dependencias inyectables**: creás una función que verifica el JWT, extrae el rol, y lanza 403 si el rol no alcanza. Esa función se pasa como `Depends()` en cada endpoint que lo requiera.

1.3 — Unit of Work (UoW)

El **Unit of Work** es el patrón que agrupa varias operaciones de base de datos en una sola transacción atómica. **Atómica** significa que o todas tienen éxito, o ninguna se aplica (rollback automático).

¿Por qué lo necesitás para pedidos? Imaginá que al crear un pedido tenés que:

1. Insertar un registro en la tabla Pedido
2. Insertar N registros en DetallePedido (uno por producto)

3. Insertar el primer registro en HistorialEstadoPedido
4. Actualizar el stock de cada producto

Si el paso 3 falla después de que el 1 y 2 ya se ejecutaron, la base de datos queda en un estado inconsistente. El UoW previene eso: si cualquier paso falla, todos los anteriores se revierten. La implementación usa un **context manager** de Python:

```
# app/uow.py
from contextlib import contextmanager
from sqlmodel import Session
from .database import engine

class UnitOfWork:
    def __init__(self):
        self.session: Session = Session(engine)

    def __enter__(self):
        return self # al entrar al 'with', devuelve el UoW

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type: # si hubo cualquier excepción...
            self.session.rollback() # ...revertí todo
        else:
            self.session.commit() # ...si no, confirmá
            self.session.close()

# Uso en el servicio:
def crear_pedido(data: PedidoCreate, user_id: int):
    with UnitOfWork() as uow:
        pedido = Pedido(usuario_id=user_id, ...)
        uow.session.add(pedido)
        # si esto falla, el pedido también se revierte:
        for item in data.items:
            detalle = DetallePedido(pedido_id=pedido.id, ...)
            uow.session.add(detalle)
        # el commit se hace solo al salir del 'with' si no hubo errores
```



El service NUNCA llama a `session.commit()` directamente. Eso es responsabilidad del UoW. Esta separación hace que la lógica de negocio sea independiente de cómo se gestiona la transacción.

1.4 — Repository Pattern

El **Repository Pattern** es una capa de abstracción entre la lógica de negocio y la base de datos. En lugar de que el service escriba queries directamente, llama a métodos del repository como `repo.get_by_id(5)` o `repo.find_by_email('x@y.com')`. La ventaja es que podés cambiar la DB o el ORM sin tocar los services.

```
# app/repositories/base_repository.py
from typing import Generic, TypeVar, Type, Optional
from sqlmodel import Session, select
from sqlmodel import SQLModel

T = TypeVar('T', bound=SQLModel)
```

```

class BaseRepository(Generic[T]):
    def __init__(self, model: Type[T], session: Session):
        self.model = model
        self.session = session

    def get_by_id(self, id: int) -> Optional[T]:
        return self.session.get(self.model, id)

    def get_all(self, skip: int = 0, limit: int = 20) -> list[T]:
        return
self.session.exec(select(self.model).offset(skip).limit(limit)).all()

    def add(self, obj: T) -> T:
        self.session.add(obj)
        self.session.flush() # flush: aplica el SQL sin hacer commit
        self.session.refresh(obj)
        return obj

    def delete(self, obj: T) -> None:
        self.session.delete(obj)

# Extensión específica:
class UsuarioRepository(BaseRepository[Usuario]):
    def __init__(self, session: Session):
        super().__init__(Usuario, session)

    def get_by_email(self, email: str) -> Optional[Usuario]:
        return self.session.exec(
            select(Usuario).where(Usuario.email == email)
        ).first()

```

 *flush() aplica el SQL a la transacción actual (para que el ID esté disponible) pero no hace commit. El commit lo hace el UoW al salir del bloque 'with'. Esto es fundamental para la operación atómica de crear un pedido con sus detalles.*

1.5 — Máquina de estados del Pedido

Los pedidos no pueden ir a cualquier estado: un pedido ENTREGADO no puede volver a EN_PREP. Esto se llama **máquina de estados finitos**. Las transiciones válidas son:

```

PENDIENTE → CONFIRMADO → EN_PREP → EN_CAMINO → ENTREGADO
      ↓           ↓
    CANCELADO   CANCELADO

```

Regla: el CLIENT solo puede cancelar desde PENDIENTE o CONFIRMADO.
El rol PEDIDOS puede avanzar desde CONFIRMADO hasta ENTREGADO.

Implementás la validación en la capa de servicio (nunca en el router):

```

# app/services/pedido_service.py
TRANSICIONES_VALIDAS = {
    'PENDIENTE': ['CONFIRMADO', 'CANCELADO'],
    'CONFIRMADO': ['EN_PREP', 'CANCELADO'],
    'EN_PREP': ['EN_CAMINO'],
    'EN_CAMINO': ['ENTREGADO'],
}

```

```

'ENTREGADO': [], # estado final
'CANCELADO': [], # estado final
}

def avanzar_estado(uow: UnitOfWork, pedido_id: int,
                  nuevo_estado: str, usuario_id: int):
    repo = PedidoRepository(uow.session)
    pedido = repo.get_by_id(pedido_id)
    if not pedido:
        raise HTTPException(404, 'Pedido no encontrado')

    estado_actual = pedido.estado_actual # campo calculado o FK
    if nuevo_estado not in TRANSICIONES_VALIDAS[estado_actual]:
        raise HTTPException(422,
            f'No se puede pasar de {estado_actual} a {nuevo_estado}')

    # Snapshot: registramos el cambio en el historial (solo INSERT)
    historial = HistorialEstadoPedido(
        pedido_id=pedido_id,
        estado=nuevo_estado,
        cambiado_por=usuario_id
    )
    uow.session.add(historial)
    pedido.estado_actual = nuevo_estado
    uow.session.add(pedido)

```

1.6 — Snapshot Pattern en DetallePedido

El **Snapshot Pattern** significa que al crear el pedido, **copiás el precio y nombre del producto en DetallePedido**, en lugar de guardar solo la FK. ¿Por qué? Porque si mañana el ADMIN cambia el precio del producto, los pedidos históricos deben conservar el precio que tenían al momento de la compra.

```

# app/models/detalle_pedido.py
class DetallePedido(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    pedido_id: int = Field(foreign_key='pedido.id')
    producto_id: int = Field(foreign_key='producto.id')
    cantidad: int = Field(ge=1)

    # SNAPSHOT: valores copiados al momento de crear el pedido
    # Son inmutables: nunca se actualizan aunque el producto cambie
    precio_snapshot: float # precio del producto en ese momento
    nombre_snapshot: str # nombre del producto en ese momento

    # Relación hacia el producto (para mostrar foto, etc.)
    producto: Optional['Producto'] = Relationship()

```

 En el service, cuando creás el `DetallePedido`, hacés: `precio_snapshot=producto.precio_base`, `nombre_snapshot=producto.nombre`. Así, aunque el producto cambie de nombre o precio, el historial queda intacto.

PARTE 2 — Implementación del Backend Nuevo

2.1 — Nuevos modelos SQLAlchemy

Estos son los modelos que el Parcial 2 agrega. Los del Parcial 1 (Producto, Categoría, Ingrediente) no cambian.

Usuario y Rol

```
# app/models/usuario.py
class Usuario(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    nombre: str = Field(max_length=100)
    email: str = Field(unique=True, max_length=150)
    password_hash: str # NUNCA guardar la contraseña en texto plano
    activo: bool = Field(default=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    deleted_at: Optional[datetime] = None

    # Relaciones
    roles: list['UsuarioRol'] = Relationship(back_populates='usuario')
    pedidos: list['Pedido'] = Relationship(back_populates='usuario')
    direcciones: list['DireccionEntrega'] = Relationship(back_populates='usuario')

# app/models/rol.py
class Rol(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    # Usamos el código como identificador semántico
    codigo: str = Field(unique=True, max_length=20) # 'ADMIN', 'CLIENT', etc.
    descripcion: Optional[str] = None
    usuario_links: list['UsuarioRol'] = Relationship(back_populates='rol')

# app/models/usuario_rol.py (tabla N:N)
class UsuarioRol(SQLModel, table=True):
    usuario_id: int = Field(foreign_key='usuario.id', primary_key=True)
    rol_id: int = Field(foreign_key='rol.id', primary_key=True)
    usuario: Optional[Usuario] = Relationship(back_populates='roles')
    rol: Optional[Rol] = Relationship(back_populates='usuario_links')
```

Pedido y DetallePedido

```
# app/models/pedido.py
class Pedido(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    usuario_id: int = Field(foreign_key='usuario.id')
    direccion_id: int = Field(foreign_key='direccionentrega.id')
    forma_pago_id: int = Field(foreign_key='formapago.id')
    estado_actual: str = Field(default='PENDIENTE') # desnormalizado para queries rápidas
    total: float # calculado al crear, no se actualiza
    created_at: datetime = Field(default_factory=datetime.utcnow)
    deleted_at: Optional[datetime] = None

    usuario: Optional[Usuario] = Relationship(back_populates='pedidos')
    detalles: list['DetallePedido'] = Relationship(back_populates='pedido')
    historial: list['HistorialEstadoPedido'] = Relationship(back_populates='pedido')
```

```
# app/models/historial_estado_pedido.py
class HistorialEstadoPedido(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    pedido_id: int = Field(foreign_key='pedido.id')
    estado: str # el nuevo estado al que pasó
    cambiado_por: int = Field(foreign_key='usuario.id')
    fecha: datetime = Field(default_factory=datetime.utcnow)
    # APPEND-ONLY: nunca UPDATE ni DELETE sobre esta tabla
    pedido: Optional[Pedido] = Relationship(back_populates='historial')
```

2.2 — Autenticación: registro, login y /me

Hash de contraseñas con bcrypt

Nunca guardás la contraseña real. bcrypt genera un hash unidireccional. Al verificar, rehacheás lo que el usuario ingresó y comparás con lo guardado. **El cost factor ≥ 12** lo define el parcial como mínimo (a mayor factor, más difícil de crackear por fuerza bruta).

```
# pip install bcrypt
import bcrypt

def hash_password(plain: str) -> str:
    # cost factor=12: balance entre seguridad y velocidad de hashing
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt(rounds=12)).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())
```

Generar y verificar JWT

```
# pip install PyJWT
import jwt
from datetime import datetime, timedelta
import os

SECRET_KEY = os.getenv('JWT_SECRET') # clave secreta en .env
ALGORITHM = 'HS256'
EXPIRE_MINUTES = 30

def create_access_token(data: dict) -> str:
    payload = data.copy()
    payload['exp'] = datetime.utcnow() + timedelta(minutes=EXPIRE_MINUTES)
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except jwt.ExpiredSignatureError:
        raise HTTPException(401, 'Token expirado')
    except jwt.InvalidTokenError:
        raise HTTPException(401, 'Token inválido')
```

Endpoints de auth y cookie HttpOnly

```
# app/routers/auth.py
from fastapi import APIRouter, Depends, Response, Request
from fastapi.security import OAuth2PasswordRequestForm

router = APIRouter(prefix='/api/v1/auth', tags=['Auth'])

@router.post('/registro', status_code=201)
def registro(data: UsuarioCreate, session=Depends(get_session)):
    # 1. Verificar que el email no exista
    repo = UsuarioRepository(session)
    if repo.get_by_email(data.email):
        raise HTTPException(409, 'Email ya registrado')
    # 2. Hashear contraseña
    usuario = Usuario(nombre=data.nombre, email=data.email,
                      password_hash=hash_password(data.password))
    session.add(usuario)
    session.flush() # genera el id
    # 3. Asignar rol CLIENT automáticamente
    rol_client = session.exec(select(Rol).where(Rol.codigo == 'CLIENT')).first()
    session.add(UsuarioRol(usuario_id=usuario.id, rol_id=rol_client.id))
    session.commit()
    return {'mensaje': 'Usuario creado correctamente'}

@router.post('/login')
def login(form: OAuth2PasswordRequestForm = Depends(),
         response: Response = None,
         session=Depends(get_session)):
    repo = UsuarioRepository(session)
    usuario = repo.get_by_email(form.username) # OAuth2 usa 'username'
    if not usuario or not verify_password(form.password, usuario.password_hash):
        raise HTTPException(401, 'Credenciales incorrectas')
    token = create_access_token({'sub': str(usuario.id), 'rol': get_rol(usuario)})
    # Guardar en cookie HttpOnly: JS del browser NO puede leerla
    response.set_cookie(
        key='access_token',
        value=token,
        httponly=True,      # no accesible desde JS
        max_age=1800,      # 30 minutos en segundos
        samesite='lax',    # protección CSRF básica
    )
    return {'mensaje': 'Login exitoso'}

@router.get('/me')
def get_me(usuario=Depends(get_current_user)):
    return usuario # get_current_user es la dependencia que verifica el JWT
```

2.3 — Dependencias de autenticación y autorización

La función `get_current_user` es la pieza central del sistema de auth. FastAPI la ejecuta automáticamente en cada endpoint que la declare como `Depends()`. Lee la cookie, decodifica el JWT, y devuelve el usuario autenticado:

```
# app/dependencias/auth.py
from fastapi import Depends, HTTPException, Request
from sqlmodel import Session
```

```
def get_current_user(request: Request, session=Depends(get_session)):
    token = request.cookies.get('access_token')
    if not token:
        raise HTTPException(401, 'No autenticado')
    payload = decode_token(token) # lanza 401 si expiró o es inválido
    user_id = int(payload['sub'])
    usuario = session.get(Usuario, user_id)
    if not usuario or not usuario.activo:
        raise HTTPException(401, 'Usuario inactivo o no existe')
    return usuario

# Factory que genera una dependencia para un rol específico
def require_rol(*roles: str):
    def checker(usuario=Depends(get_current_user)):
        codigos = {ur.rol.codigo for ur in usuario.roles}
        if not codigos.intersection(roles):
            raise HTTPException(403, 'Sin permisos para esta acción')
        return usuario
    return checker

# Uso en los routers:
# @router.delete('/{id}', dependencies=[Depends(require_rol('ADMIN'))])
# @router.patch('/{id}/disponibilidad',
# dependencies=[Depends(require_rol('ADMIN', 'STOCK'))])
```

2.4 — Seed obligatorio

El seed es un script que **pre-popula la base de datos con datos iniciales obligatorios**: roles, estados de pedido, formas de pago, y un usuario admin. Se ejecuta una vez al inicializar el entorno. La forma más limpia es como un archivo separado o como un comando CLI:

```
# app/db/seed.py
from sqlmodel import Session, select
from ..database import engine
from ..models.rol import Rol
from ..models.usuario import Usuario
from ..models.usuario_rol import UsuarioRol
from ..models.forma_pago import FormaPago
from ..models.estado_pedido import EstadoPedido
from ..utils.security import hash_password

def seed():
    with Session(engine) as session:
        # --- Roles ---
        roles_data = [
            ('ADMIN', 'Administrador'),
            ('STOCK', 'Gestor de Stock'),
            ('PEDIDOS', 'Gestor de Pedidos'),
            ('CLIENT', 'Cliente'),
        ]
        for codigo, desc in roles_data:
            # Idempotente: solo inserta si no existe
            if not session.exec(select(Rol).where(Rol.codigo == codigo)).first():
                session.add(Rol(codigo=codigo, descripcion=desc))

        # --- Formas de pago ---
```

```

    for nombre in ['Efectivo', 'Tarjeta de crédito', 'Transferencia', 'Mercado
Pago']:
        if not session.exec(select(FormaPago).where(FormaPago.nombre ==
nombre)).first():
            session.add(FormaPago(nombre=nombre))

    # --- Estado de Pedido (catálogo de estados válidos) ---
    for codigo in
['PENDIENTE', 'CONFIRMADO', 'EN_PREP', 'EN_CAMINO', 'ENTREGADO', 'CANCELADO']:
        if not session.exec(select(EstadoPedido).where(EstadoPedido.codigo ==
codigo)).first():
            session.add(EstadoPedido(codigo=codigo))

    session.commit() # guardamos roles y formas de pago antes del admin

    # --- Usuario admin por defecto ---
    if not session.exec(select(Usuario).where(Usuario.email ==
'admin@store.com')).first():
        admin = Usuario(nombre='Admin', email='admin@store.com',
                        password_hash=hash_password('admin1234'))
        session.add(admin)
        session.flush()
        rol_admin = session.exec(select(Rol).where(Rol.codigo ==
'ADMIN')).first()
        session.add(UsuarioRol(usuario_id=admin.id, rol_id=rol_admin.id))

    session.commit()
    print('Seed completado correctamente.')

if __name__ == '__main__':
    seed()

```

 **El seed debe ser IDEMPOTENTE: si lo ejecutás dos veces, no debe duplicar datos. Por eso cada inserción verifica primero si el registro ya existe. Esto lo demostrarás en el video.**

PARTE 3 — Frontend Store (Zustand + Carrito + TanStack Query)

El frontend Store es la app pública que usa el cliente para navegar el catálogo y hacer pedidos. El gran elemento nuevo respecto al Parcial 1 es el **carrito con persistencia en localStorage** usando Zustand con el `middleware persist`, y la instancia de Axios con interceptores.

3.1 — Estructura del proyecto Store

```
store-frontend/
├── src/
│   ├── api/
│   │   └── axiosInstance.ts      # instancia de Axios con baseUrl e interceptors
│   ├── store/
│   │   └── cartStore.ts          # Zustand: carrito con middleware persist
│   ├── pages/
│   │   ├── HomePage.tsx         # catálogo de productos
│   │   ├── CartPage.tsx         # carrito y resumen
│   │   └── OrdersPage.tsx       # mis pedidos
│   ├── components/
│   │   ├── ProductCard.tsx
│   │   └── CartItem.tsx
│   └── App.tsx                  # rutas
```

3.2 — Instancia de Axios e Interceptores

Una **instancia de Axios** es una configuración preestablecida con la URL base del backend y opciones comunes. Los **interceptores** son funciones que se ejecutan automáticamente antes de cada request (interceptor de request) o después de cada respuesta (interceptor de response). Son el equivalente de un middleware pero en el cliente.

```
// src/api/axiosInstance.ts (Frontend Store)
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:8000/api/v1',
  withCredentials: true, // CRÍTICO: envía la cookie en cada request
})

// Interceptor de request: se ejecuta ANTES de cada petición
api.interceptors.request.use(
  (config) => {
    // Aquí podés agregar headers, tokens, logs, etc.
    console.log(`[API] ${config.method?.toUpperCase()} ${config.url}`)
    return config // siempre devolvé el config modificado
  },
  (error) => Promise.reject(error)
)

// Interceptor de response: se ejecuta DESPUÉS de cada respuesta
api.interceptors.response.use(
  (response) => response, // si fue exitoso, devolvé la response tal cual
  (error) => {
    // Manejo global de errores
    if (error.response?.status === 401) {
```

```

        // Si el token expiró, redirigir al login
        window.location.href = '/login'
    }
    return Promise.reject(error)
}
)

export default api

```

 *withCredentials: true es obligatorio para que el browser envíe las cookies al hacer requests. Sin esto, la cookie HttpOnly con el JWT nunca llegaría al backend y todos los endpoints protegidos devolverían 401.*

3.3 — Carrito con Zustand y middleware persist

Zustand es una librería de manejo de estado global mucho más simple que Redux. El middleware `persist` serializa automáticamente el estado del store en **localStorage**, y lo rehidrata cuando el usuario recarga la página. Así el carrito no se pierde al cerrar el tab.

```

# Instalación:
# npm install zustand

// src/store/cartStore.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { Producto } from '../types/producto'

interface CartItem {
  producto: Producto
  cantidad: number
}

interface CartStore {
  items: CartItem[]
  addItem: (producto: Producto) => void
  removeItem: (productId: number) => void
  updateCantidad: (productId: number, cantidad: number) => void
  clearCart: () => void
  total: () => number
}

export const useCartStore = create<CartStore>() ( // los () dobles son por
TypeScript
  persist(
    (set, get) => ({
      items: [],

      addItem: (producto) => set((state) => {
        const existe = state.items.find(i => i.producto.id === producto.id)
        if (existe) {
          // Si ya está, incrementa la cantidad
          return { items: state.items.map(i =>
            i.producto.id === producto.id
              ? { ...i, cantidad: i.cantidad + 1 }
              : i
          ) }
        }
      })
    })
  )
)

```

```

        return { items: [...state.items, { producto, cantidad: 1 }] }
    }},

    removeItem: (id) => set((state) => ({
        items: state.items.filter(i => i.producto.id !== id)
    })),

    updateCantidad: (id, cantidad) => set((state) => ({
        items: state.items.map(i =>
            i.producto.id === id ? { ...i, cantidad } : i
        )
    })),

    clearCart: () => set({ items: [] }),

    // 'total' es una función derivada, no estado guardado
    total: () => get().items.reduce(
        (acc, item) => acc + item.producto.precio_base * item.cantidad, 0
    ),
  }},
  {
    name: 'cart-storage', // key en localStorage
  }
)
)

```

 El persist middleware usa `JSON.stringify/JSON.parse` internamente. El estado se guarda en `localStorage` bajo la key `'cart-storage'`. Podés verlo en el devtools del browser → `Application` → `Local Storage`.

3.4 — Flujo de realizar un pedido

Al confirmar el pedido desde `CartPage`, usás `useMutation` para enviar los datos al backend:

```

// src/pages/CartPage.tsx (fragmento del botón 'Confirmar Pedido')
import { useMutation } from '@tanstack/react-query'
import { useCartStore } from '../store/cartStore'
import api from '../api/axiosInstance'

function CartPage() {
  const { items, total, clearCart } = useCartStore()

  const crearPedidoMutation = useMutation({
    mutationFn: async (pedidoData: PedidoCreate) => {
      const { data } = await api.post('/pedidos', pedidoData)
      return data
    },
    onSuccess: () => {
      clearCart() // limpia el carrito después del pedido exitoso
      navigate('/mis-pedidos')
    },
    onError: (error) => {
      // Mostrar error al usuario
    }
  })

  const handleConfirmar = () => {

```

```
    crearPedidoMutation.mutate({
      items: items.map(i => ({
        producto_id: i.producto.id,
        cantidad: i.cantidad,
      })),
      direccion_id: selectedDireccion,
      forma_pago_id: selectedFormaPago,
    })
  }
}
```

PARTE 4 — Frontend Administración (Auth + RBAC + Rutas Protegidas)

4.1 — Estructura del proyecto Admin

```
admin-frontend/
├── src/
│   ├── api/
│   │   └── axiosInstance.ts      # instancia separada (misma lógica, mismo
└── backend)
    ├── context/
    │   └── AuthContext.tsx      # estado global del usuario autenticado
    ├── pages/
    │   ├── LoginPage.tsx
    │   ├── DashboardPage.tsx
    │   ├── CategoríasPage.tsx   # del Parcial 1
    │   ├── ProductosPage.tsx   # del Parcial 1
    │   ├── IngredientesPage.tsx # del Parcial 1
    │   └── CajeroPedidosPage.tsx # nuevo: cambiar estados
    ├── components/
    │   ├── ProtectedRoute.tsx   # HOC para rutas protegidas
    │   └── RoleGuard.tsx        # componente para ocultar UI por rol
    └── App.tsx
```

4.2 — AuthContext: estado global del usuario

El AuthContext almacena el usuario logueado en el estado de React y lo comparte con toda la app. Al hacer login, llama a `/auth/me` para obtener los datos del usuario desde el backend (la cookie ya se guardó automáticamente):

```
// src/context/AuthContext.tsx
import { createContext, useContext, useState, useEffect, ReactNode } from 'react'
import api from '../api/axiosInstance'

interface AuthContextType {
  usuario: Usuario | null
  isLoading: boolean
  login: (email: string, password: string) => Promise<void>
  logout: () => void
  hasRole: (rol: string) => boolean
}

const AuthContext = createContext<AuthContextType | null>(null)

export function AuthProvider({ children }: { children: ReactNode }) {
  const [usuario, setUsuario] = useState<Usuario | null>(null)
  const [isLoading, setIsLoading] = useState(true)

  // Al montar, verifica si hay una sesión activa (cookie válida)
  useEffect(() => {
    api.get('/auth/me')
      .then(res => setUsuario(res.data))
      .catch(() => setUsuario(null)) // si no hay cookie o expiró
      .finally(() => setIsLoading(false))
  }, [])
}
```

```

    }, [])

    const login = async (email: string, password: string) => {
      // El backend setea la cookie automáticamente en la response
      await api.post('/auth/login', { username: email, password })
      const { data } = await api.get('/auth/me')
      setUsuario(data)
    }

    const logout = async () => {
      await api.post('/auth/logout') // el backend limpia la cookie
      setUsuario(null)
    }

    const hasRole = (rol: string) =>
      usuario?.roles?.some((ur: any) => ur.rol.codigo === rol) ?? false

    return (
      <AuthContext.Provider value={{ usuario, isLoading, login, logout, hasRole }}>
        {children}
      </AuthContext.Provider>
    )
  }

  // Hook custom para usar el contexto fácilmente
  export const useAuth = () => {
    const ctx = useContext(AuthContext)
    if (!ctx) throw new Error('useAuth debe usarse dentro de AuthProvider')
    return ctx
  }

```

4.3 — Rutas protegidas por autenticación y rol

```

// src/components/ProtectedRoute.tsx
import { Navigate } from 'react-router-dom'
import { useAuth } from '../context/AuthContext'

interface Props {
  children: React.ReactNode
  roles?: string[] // si se especifican, verifica el rol también
}

export function ProtectedRoute({ children, roles }: Props) {
  const { usuario, isLoading } = useAuth()

  if (isLoading) return <div>Cargando...</div>

  // Si no está autenticado, redirigir al login
  if (!usuario) return <Navigate to='/login' replace />

  // Si se requieren roles y el usuario no los tiene
  if (roles && !roles.some(r => usuario.roles?.some((ur: any) => ur.rol.codigo === r))) {
    return <Navigate to='/no-autorizado' replace />
  }

```

```

    }

    return <>{children}</>
  }

// Uso en App.tsx:
// <Route path='/admin/productos' element={
//   <ProtectedRoute roles={['ADMIN']}>
//     <ProductosPage />
//   </ProtectedRoute>
// } />
// <Route path='/cajero/pedidos' element={
//   <ProtectedRoute roles={['ADMIN', 'PEDIDOS']}>
//     <CajeroPedidosPage />
//   </ProtectedRoute>
// } />

```

4.4 — Modo Admin vs Modo Empleado en la UI

El parcial pide que ADMIN pueda hacer CRUD completo y que el rol PEDIDOS/empleado solo vea. En lugar de duplicar páginas, usás el hook `useAuth` para **mostrar u ocultar botones y acciones** según el rol:

```

// Ejemplo en ProductosPage.tsx
import { useAuth } from '../context/AuthContext'

function ProductosPage() {
  const { hasRole } = useAuth()
  const esAdmin = hasRole('ADMIN')

  return (
    <div>
      {/* Solo el ADMIN ve el botón de crear */}
      {esAdmin && (
        <button onClick={() => setModalOpen(true)}>
          + Nuevo Producto
        </button>
      )}

      <table>
        {/* ... */}
        <td>
          {esAdmin ? (
            <>
              <button>Editar</button>
              <button>Eliminar</button>
            </>
          ) : (
            <span className='text-gray-400 text-sm'>Solo lectura</span>
          )}
        </td>
      </table>
    </div>
  )
}

```

4.5 — Pantalla del Cajero: cambiar estados de pedidos

```
// src/pages/CajeroPedidosPage.tsx
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'
import api from '../api/axiosInstance'

// Los estados siguientes válidos para cada estado actual
const SIGUIENTES_ESTADOS: Record<string, string[]> = {
  PENDIENTE: ['CONFIRMADO', 'CANCELADO'],
  CONFIRMADO: ['EN_PREP', 'CANCELADO'],
  EN_PREP: ['EN_CAMINO'],
  EN_CAMINO: ['ENTREGADO'],
  ENTREGADO: [],
  CANCELADO: [],
}

function CajeroPedidosPage() {
  const queryClient = useQueryClient()

  const { data: pedidos } = useQuery({
    queryKey: ['pedidos'],
    queryFn: () => api.get('/pedidos').then(r => r.data),
    refetchInterval: 30000, // refresca cada 30s automáticamente
  })

  const avanzarMutation = useMutation({
    mutationFn: ({ pedidoId, nuevoEstado }: { pedidoId: number, nuevoEstado:
string }) =>
      api.patch(`/pedidos/${pedidoId}/estado`, { nuevo_estado: nuevoEstado
}),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['pedidos'] })
  })

  return (
    <div className='p-6'>
      <h1 className='text-2xl font-bold mb-4'>Panel de Pedidos</h1>
      {pedidos?.map((pedido: any) => (
        <div key={pedido.id} className='bg-white rounded-lg shadow p-4
mb-3'>
          <div className='flex justify-between items-center'>
            <div>
              <span className='font-semibold'>Pedido
#{pedido.id}</span>
              <span className='ml-3 bg-blue-100 text-blue-700 px-2
py-1 rounded text-sm'>
                {pedido.estado_actual}
              </span>
            </div>
            <div className='flex gap-2'>
              {SIGUIENTES_ESTADOS[pedido.estado_actual]?.map(siguiete => (
                <button
                  key={siguiete}
                  onClick={() => avanzarMutation.mutate({
                    pedidoId: pedido.id,
                    nuevoEstado: siguiete
                  })}
                  className='bg-green-600 text-white px-3 py-1
rounded text-sm hover:bg-green-700'
                >
```

```
        → {siguiente}
      </button>
    )}
  </div>
</div>
</div>
  )}
</div>
)
}
```

PARTE 5 — Plan de Trabajo, Orden de Implementación y Tips

5.1 — Orden recomendado para implementar (haciendo todo solo)

Como vas a hacer todo vos solo, el orden importa mucho para no bloquearte. El backend primero, siempre:

5. **Backend: modelos nuevos + seed.** Creá los modelos (Usuario, Rol, Pedido, etc.), corrés el seed y verificás en pgAdmin que las tablas y datos estén. Esto desbloquea todo lo demás.
6. **Backend: auth (registro + login + /me).** Antes de cualquier frontend, verificá en Swagger (/docs) que podés registrarte, loguearte y que la cookie se setea correctamente.
7. **Backend: RBAC + endpoints de pedidos.** Con la auth funcionando, agregás los decoradores de rol a los endpoints del Parcial 1 y construí el CRUD de pedidos con la máquina de estados.
8. **Frontend Store: instancia Axios + catálogo + carrito.** Empezás por el carrito con Zustand (funciona offline), después conectás con el backend.
9. **Frontend Store: flujo de pedido.** Con el carrito funcionando, agregás la pantalla de confirmar pedido.
10. **Frontend Admin: login + AuthContext + rutas protegidas.** Primero que funcione el login y la redirección. Después portás las páginas del Parcial 1.
11. **Frontend Admin: pantalla cajero.** Último porque depende de que los pedidos existan en la DB.

5.2 — Errores comunes específicos del Parcial 2

- **Cookie no llega al backend:** Verificá que withCredentials: true esté en la instancia de Axios Y que el CORSMiddleware tenga allow_credentials=True y allow_origins con la URL exacta del frontend (no '*' cuando hay credentials).
- **JWT expirado en el /me inicial:** El useEffect del AuthContext llama a /me al montar. Si el token ya expiró, el 401 no debe crashear la app, solo setear usuario=null. El interceptor de Axios maneja esto.
- **flush() vs commit() en el UoW:** Usá flush() cuando necesitás el ID generado por la DB (ej: para usarlo como FK en el siguiente insert) pero aún no querés hacer commit. El commit lo hace el UoW al salir del bloque.
- **Estado de pedido no sincronizado:** El campo estado_actual en Pedido es una desnormalización. Cuando avanzás el estado, actualizá tanto ese campo como el HistorialEstadoPedido en la misma transacción.
- **CORS con dos frontends:** Agregá ambos puertos al allow_origins del backend: ['http://localhost:5173', 'http://localhost:5174'].

5.3 — Checklist antes de grabar el video

- **Backend:** Seed corre sin errores. Registro y login funcionan en /docs. El JWT se setea como cookie. Los endpoints de pedidos respetan la máquina de estados.
- **Store:** El carrito persiste al recargar la página. El flujo completo de pedido (agregar al carrito → confirmar → ver en 'mis pedidos') funciona.

- **Admin:** El login redirige al dashboard. Rutas protegidas redirigen a /login si no hay sesión. El cajero puede avanzar estados y la lista se actualiza.
- **RBAC en la UI:** Logueate como ADMIN y verificá que se ven los botones de editar/eliminar. Logueate como rol PEDIDOS y verificá que solo ve la lista.

Reflexión final

El Parcial 2 tiene más piezas móviles que el 1, pero conceptualmente sigue los mismos principios: **separación de responsabilidades, transacciones atómicas, y estado del servidor sincronizado con la UI**. El JWT, el UoW y el persist de Zustand son herramientas concretas para resolver problemas concretos: autenticación segura, integridad de datos, y persistencia del carrito.

Si en el video podés explicar POR QUÉ existe cada patrón (no solo mostrar que funciona), eso es lo que diferencia una nota de 7 de una de 10.

¡Éxitos en el parcial! — Fecha de entrega: 25/05/26.